

Experiment No.6

Title : Program to implement Exception and Custom exception in java

Name of student : Shreya Dipak Patil

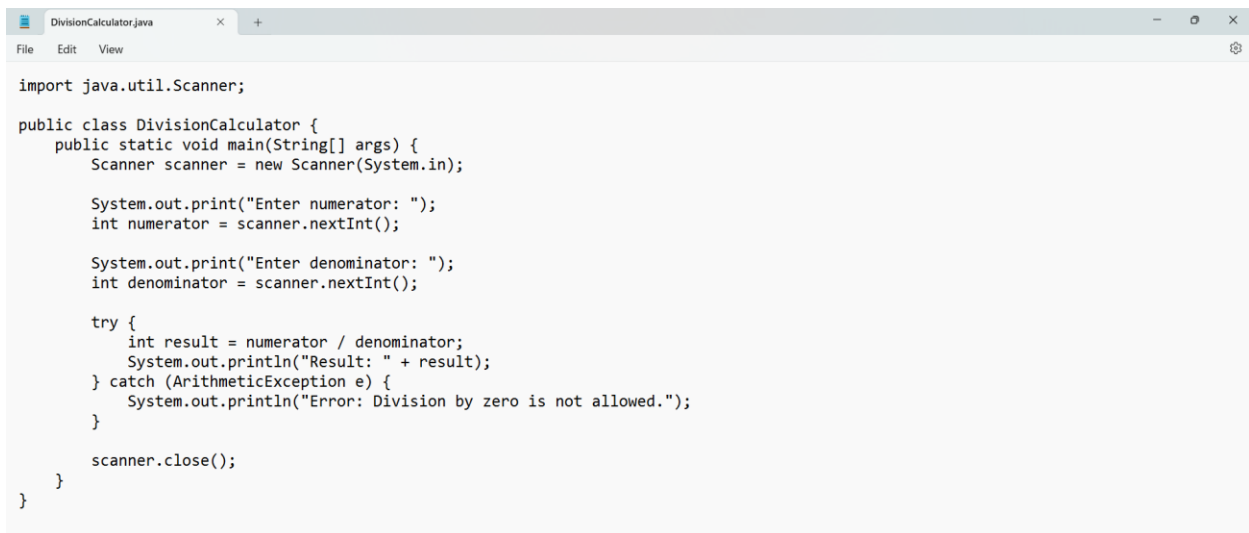
Roll No.2015

URN: 23031038

Class : SY A

1. Problem Statement:

You are creating a simple calculator that divides two numbers. The user will input two integers. However, there is a risk that the denominator may be zero, which would lead to a division by zero error. You need to handle the `ArithmeticException` when this happens.

A screenshot of an IDE window titled 'DivisionCalculator.java'. The code defines a class 'DivisionCalculator' with a 'main' method. It uses a 'Scanner' to take input from the user. It prompts for a numerator and a denominator. A try-catch block is used to handle the 'ArithmeticException' that occurs when the denominator is zero. In this case, it prints an error message: 'Error: Division by zero is not allowed.'.

```
import java.util.Scanner;

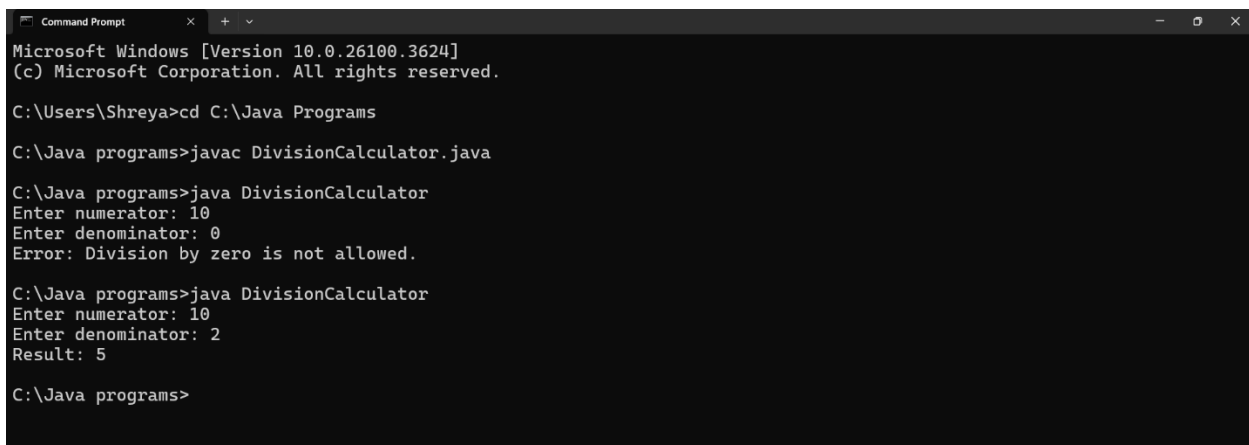
public class DivisionCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter numerator: ");
        int numerator = scanner.nextInt();

        System.out.print("Enter denominator: ");
        int denominator = scanner.nextInt();

        try {
            int result = numerator / denominator;
            System.out.println("Result: " + result);
        } catch (ArithmeticException e) {
            System.out.println("Error: Division by zero is not allowed.");
        }

        scanner.close();
    }
}
```

A screenshot of a Windows Command Prompt. It shows the user navigating to the 'C:\Java Programs' directory and compiling the 'DivisionCalculator.java' file using 'javac'. Then, the user runs the program using 'java DivisionCalculator'. The first run shows an input of numerator 10 and denominator 0, resulting in the error message 'Error: Division by zero is not allowed.'. The second run shows an input of numerator 10 and denominator 2, resulting in the output 'Result: 5'.

```
Microsoft Windows [Version 10.0.26100.3624]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Shreya>cd C:\Java Programs

C:\Java programs>javac DivisionCalculator.java

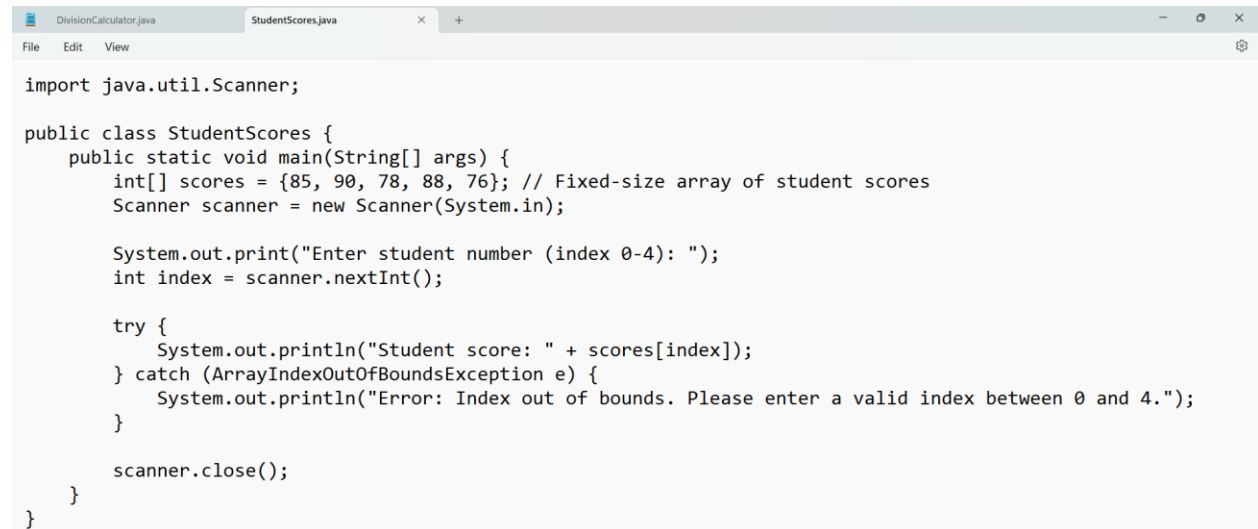
C:\Java programs>java DivisionCalculator
Enter numerator: 10
Enter denominator: 0
Error: Division by zero is not allowed.

C:\Java programs>java DivisionCalculator
Enter numerator: 10
Enter denominator: 2
Result: 5

C:\Java programs>
```

2. Problem Statement:

You are working with an array of student scores for a class. The array has a fixed size of 5. The user will input a student number (index) to retrieve the score. However, if the user enters an index outside the range of the array, it should throw an `ArrayIndexOutOfBoundsException`.

A screenshot of an IDE window titled 'StudentScores.java'. The code defines a public class 'StudentScores' with a 'main' method. It imports 'java.util.Scanner'. Inside 'main', it initializes an array 'scores' with values {85, 90, 78, 88, 76} and creates a 'Scanner' object. It prompts the user to enter a student number (index 0-4). It then uses a try-catch block to retrieve the score at the given index. If the index is out of bounds, it catches the 'ArrayIndexOutOfBoundsException' and prints an error message. Finally, it closes the scanner.

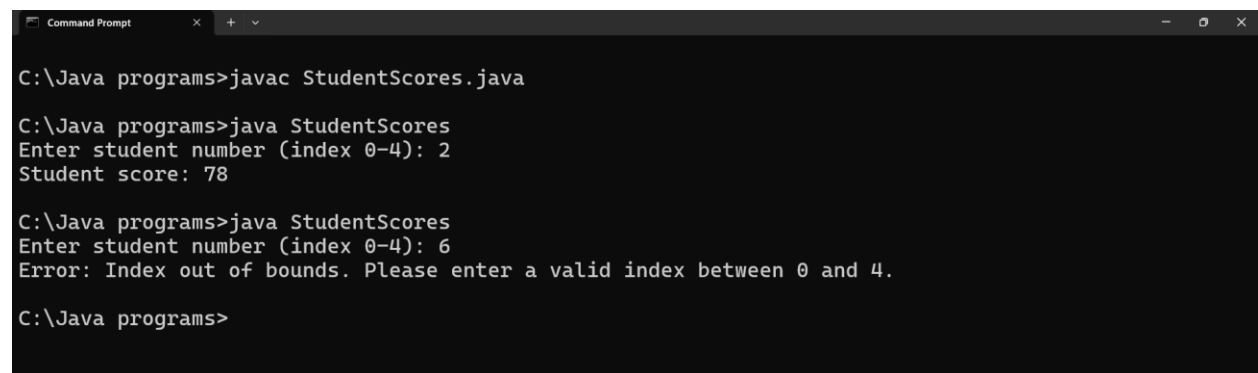
```
import java.util.Scanner;

public class StudentScores {
    public static void main(String[] args) {
        int[] scores = {85, 90, 78, 88, 76}; // Fixed-size array of student scores
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter student number (index 0-4): ");
        int index = scanner.nextInt();

        try {
            System.out.println("Student score: " + scores[index]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Error: Index out of bounds. Please enter a valid index between 0 and 4.");
        }

        scanner.close();
    }
}
```

A screenshot of a Windows Command Prompt window. It shows the compilation and execution of the 'StudentScores.java' file. The first command 'javac StudentScores.java' compiles the file. The second command 'java StudentScores' runs the program. The first run shows the user entering '2' and receiving the score '78'. The second run shows the user entering '6', which is out of bounds, resulting in an error message: 'Error: Index out of bounds. Please enter a valid index between 0 and 4.'

```
C:\Java programs>javac StudentScores.java

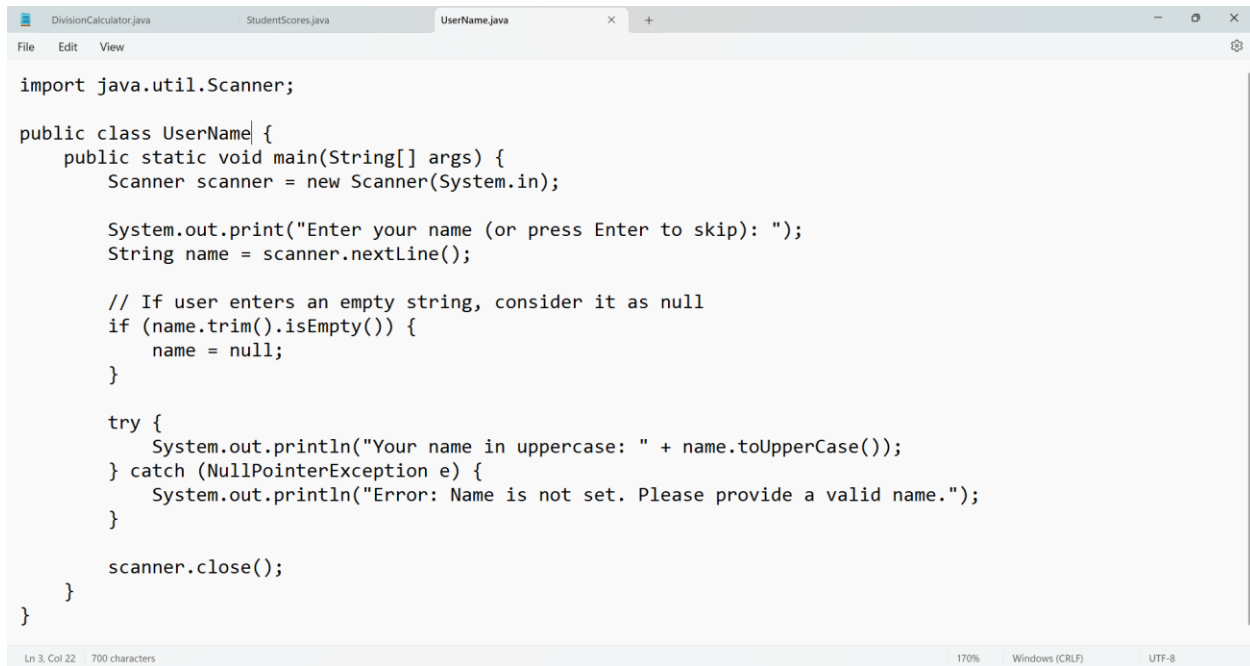
C:\Java programs>java StudentScores
Enter student number (index 0-4): 2
Student score: 78

C:\Java programs>java StudentScores
Enter student number (index 0-4): 6
Error: Index out of bounds. Please enter a valid index between 0 and 4.

C:\Java programs>
```

3. Problem Statement:

You are working on a program where you store a user's name in a `String` variable. Sometimes, the user might not have a name set (i.e., it is `null`). If you try to convert the name to uppercase without checking if it's `null`, it will throw a `NullPointerException`.

A screenshot of an IDE window with three tabs: DivisionCalculator.java, StudentScores.java, and UserName.java. The UserName.java tab is active, showing the following Java code:

```
import java.util.Scanner;

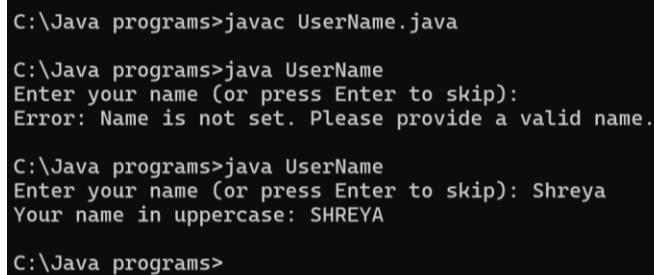
public class UserName {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your name (or press Enter to skip): ");
        String name = scanner.nextLine();

        // If user enters an empty string, consider it as null
        if (name.trim().isEmpty()) {
            name = null;
        }

        try {
            System.out.println("Your name in uppercase: " + name.toUpperCase());
        } catch (NullPointerException e) {
            System.out.println("Error: Name is not set. Please provide a valid name.");
        }

        scanner.close();
    }
}
```

The status bar at the bottom indicates "Ln 3, Col 22", "700 characters", "170%", "Windows (CRLF)", and "UTF-8".A screenshot of a Windows command prompt with a black background and white text. It shows the compilation and execution of the UserName.java program:

```
C:\Java programs>javac UserName.java

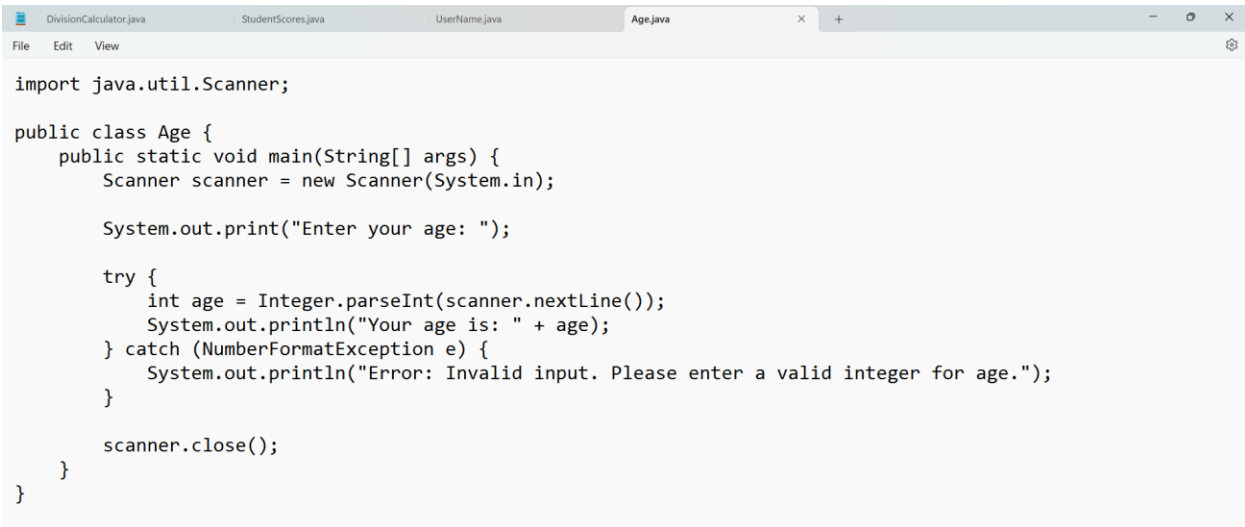
C:\Java programs>java UserName
Enter your name (or press Enter to skip):
Error: Name is not set. Please provide a valid name.

C:\Java programs>java UserName
Enter your name (or press Enter to skip): Shreya
Your name in uppercase: SHREYA

C:\Java programs>
```

4. Problem Statement:

You are writing a program that asks the user to input their age as a number. If the user enters anything that is not a valid integer (such as a string or a special character), the program should handle the `NumberFormatException`.

A screenshot of an IDE window titled 'Age.java'. The window has tabs for 'DivisionCalculator.java', 'StudentScores.java', 'UserName.java', and 'Age.java'. The menu bar shows 'File', 'Edit', and 'View'. The code is as follows:

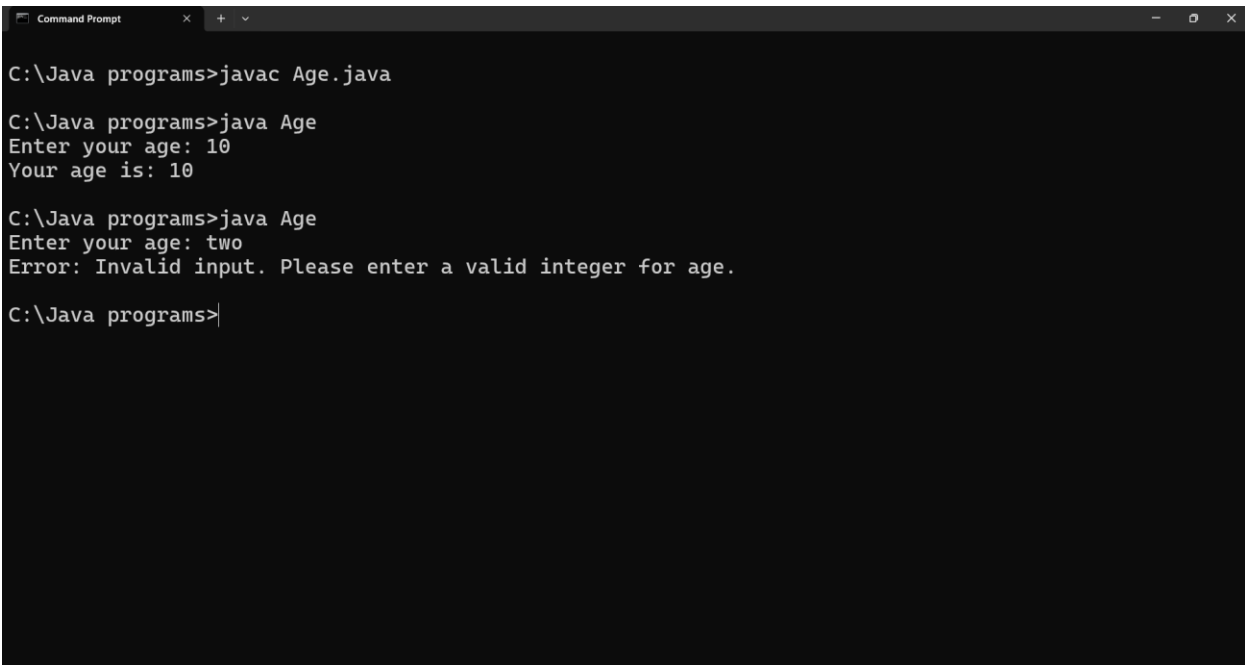
```
import java.util.Scanner;

public class Age {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter your age: ");

        try {
            int age = Integer.parseInt(scanner.nextLine());
            System.out.println("Your age is: " + age);
        } catch (NumberFormatException e) {
            System.out.println("Error: Invalid input. Please enter a valid integer for age.");
        }

        scanner.close();
    }
}
```

A screenshot of a Windows Command Prompt window. The title bar says 'Command Prompt'. The command prompt shows the following sequence of commands and outputs:

```
C:\Java programs>javac Age.java

C:\Java programs>java Age
Enter your age: 10
Your age is: 10

C:\Java programs>java Age
Enter your age: two
Error: Invalid input. Please enter a valid integer for age.

C:\Java programs>
```